

Программная реализация симплекс-метода

В программе самостоятельно реализованы выбор разрешающего столбца и строки, пересчёт симплекс-таблицы, решение вспомогательной задачи, переход к основной задаче и получение итогового решения.

Выбор разрешающего столбца и строки

```
def find_entering_column(table):  
    """Выбор переменный, которая войдет в базис"""  
  
    objective_row = table[-1][: -1]  
    min_value = min(objective_row)  
  
    if min_value >= 0:  
        return None  
  
    return objective_row.index(min_value)  
  
def find_leaving_row(table, column):  
    """Выбор переменной, которая выйдет из базиса"""  
  
    best_row = None  
    best_ratio = None  
  
    for i in range(len(table) - 1):  
        a = table[i][column]  
        b = table[i][ -1]  
  
        if a > 0:  
            ratio = b / a  
  
            if best_ratio is None or ratio < best_ratio:  
                best_ratio = ratio  
                best_row = i  
  
    return best_row
```

После выбора разрешающего столбца определяется переменная, которая должна выйти из базиса. Для положительных элементов выбранного столбца вычисляется отношение свободного члена к соответствующему коэффициенту:

$$b_i/a_{ij}$$

Из полученных значений выбирается минимальное.

После выбора разрешающей строки и столбца выполняется пересчёт симплекс-таблицы.

Сначала пересчитывается разрешающая строка. Все её элементы делятся на разрешающий элемент. Для элемента, находящегося на пересечении разрешающей строки и столбца, используется обратное значение разрешающего элемента.

Остальные строки пересчитываются по формулам симплекс-метода.

```
def pivot(table, basis, nonbasis, row, column):
    pivot_element = table[row][column]

    old_pivot_row = table[row][:]

    leaving_variable = basis[row]
    entering_variable = nonbasis[column]

    print(
        f"\nВ базис входит {entering_variable}, "
        f"из базиса выходит {leaving_variable}."
    )
    print(f"Ведущий элемент = {pivot_element}")

    # Пересчитываем ведущей строки
    new_pivot_row = [f(0) for _ in table[row]]

    for j in range(len(nonbasis)):
        if j == column:
            new_pivot_row[j] = f(1) / pivot_element
        else:
            new_pivot_row[j] = old_pivot_row[j] / pivot_element

    new_pivot_row[-1] = old_pivot_row[-1] / pivot_element

    # Пересчет остальных строк

    for i in range(len(table)):
        if i == row:
            continue

        old_row = table[i][:]
        coefficient = old_row[column]
```

```

for j in range(len(nonbasis)):
    if j == column:
        table[i][j] = -coefficient / pivot_element
    else:
        table[i][j] = (
            old_row[j]
            - coefficient * old_pivot_row[j] / pivot_element
        )

table[i][-1] = (
    old_row[-1]
    - coefficient * old_pivot_row[-1] / pivot_element
)

table[row] = new_pivot_row

basis[row] = entering_variable
nonbasis[column] = leaving_variable

```

Для автоматического выполнения всех итераций используется функция `simplex()`. Она повторяет выбор разрешающего столбца и строки и выполняет пересчёт таблицы до тех пор, пока в последней строке не останется отрицательных коэффициентов.

```

def simplex(table, basis, nonbasis, phase_name):
    """Симплекс-шаги"""
    step = 1
    while True:
        column = find_entering_column(table)
        if column is None:
            print(f"\n{phase_name}: отрицательных коэффициентов больше нет.")
            break
        row = find_leaving_row(table, column)
        if row is None:
            raise ValueError("Целевая функция не ограничена снизу.")

        pivot(table, basis, nonbasis, row, column)
        print_table(
            table,
            basis,
            nonbasis,
            f"{phase_name}. Таблица после шага {step}",
        )
        step += 1

```

Начальная симплекс-таблица для исходной таблицы

```

basis = ["x7", "x8", "x9"]
nonbasis = ["x1", "x2", "x3", "x4", "x5", "x6"]
table = [
    [f(2), f(1), f(0), f(1), f(1), f(0), f(9)],
    [f(1), f(1), f(1), f(0), f(0), f(0), f(7)],
    [f(0), f(0), f(1), f(1), f(0), f(-1), f(5)],
    [f(-3), f(-2), f(-2), f(-2), f(-1), f(1), f(-21)],
]
print_table(
    table,
    basis,
    nonbasis,
    "Начальная таблица вспомогательной задачи",
)
simplex(table, basis, nonbasis, "1")

```

После формирования начальной таблицы дальнейшие симплекс-преобразования выполняются программой автоматически.

После завершения вспомогательной задачи программа проверяет значение вспомогательной целевой функции. Если оно равно нулю, это означает, что исходная задача имеет допустимые решения.

После решения вспомогательной задачи искусственные переменные x_7, x_8, x_9 больше не нужны. Их столбцы удаляются из таблицы.

После завершения симплекс-метода небазисные переменные принимаются равными нулю, а значения базисных переменных берутся из столбца свободных членов b .

```

ОТВЕТ
x1 = 0
x2 = 2
x3 = 5
x4 = 0
Z_min = 7

Вектор решения:
x* = (0; 2; 5; 0)

```

Полученный программой результат полностью совпадает с результатом ручного решения симплекс-методом.